МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ

про виконання лабораторної роботи №6

«Автоматизоване тестування Python-програм»

з навчальної дисципліни «Професійний Python»

Варіант 4: «Система банківських рахунків»

Виконав: студент групи ФЕП-33
Зінчук Остап

Керівник: д.ф.-м.н., професор кафедри СП
Свелеба Сергій Андрійович

Львів — 2026

1. Мета роботи

Мета лабораторної роботи: Опанувати теоретичні основи та практичні інженерні навички автоматизованого тестування програмного забезпечення мовою Python із використанням фреймворку pytest, бібліотеки макетування unittest.mock (Mock, AsyncMock) та інструменту аудиту тестового покриття pytest-cov.

У процесі виконання роботи реалізується комплекс модульних, інтеграційних та дослідницьких тестів для банківської системи обліку рахунків (Варіант 4), забезпечується збереження повної спадкоємності кодової бази Лабораторної роботи №5, досліджуються механізми ізоляції ресурсів (tmp_path, monkeypatch), макетуються сценарії відмов (side_effect) та аналізується покриття розгалужень (Branch Coverage) із досягненням показника понад 90%.

2. Завдання роботи

- Створити структурований тестовий набір відповідно до архітектури проєкту (unit, integration, experiments).
- Забезпечити наявність усіх вихідних модулів та конфігураційних файлів Лабораторної роботи №5 у проєкті Лабораторної роботи №6.
- Реалізувати не менше 15 модульних тестів для моделей, валідаторів, конфігурації, експортерів та банківських сервісів.
- Реалізувати не менше 3 інтеграційних тестів для перевірки наскрізних сценаріїв імпорту, експорту та життєвого циклу додатку.
- Розробити конфігураційний модуль tests/conftest.py із щонайменше 3 фікстурами та фабрикою об'єктів.
- Застосувати параметризоване тестування (@pytest.mark.parametrize) для перевірки граничних та аномальних даних.
- Реалізувати верифікацію виняткових ситуацій за допомогою pytest.raises із перевіркою типу та атрибутів помилки.
- Використати pytest.approx для безпечного порівняння чисел із рухомою комою (фінансові баланси та суми транзакцій).
- Забезпечити ізоляцію файлової системи через стандартну фікстуру tmp_path.
- Застосувати monkeypatch для ізоляції конфігурації, середовища та аргументів CLI.
- Реалізувати макетування залежностей (сповіщувач Notifier, репозиторій AccountRepository) за допомогою Mock.
- Налаштувати поведінку дублерів через mock.return_value та емуляцію збоїв через mock.side_effect.
- Здійснити сувору перевірку протоколу взаємодії: assert_called_once_with, assert_not_called, call_count.
- Реалізувати асинхронне тестування зовнішнього банківського шлюзу за допомогою AsyncMock.
- Провести 7 обов'язкових лабораторних експериментів (Розділи 84–90 методички) та зафіксувати числові результати.
- Досягти тестового покриття розгалужень (Branch Coverage) на рівні не менше 80–90% та згенерувати HTML-звіт.
- Оформити технічний звіт згідно з 32 обов'язковими розділами та надати відповіді на всі 80 контрольних питань.

3. Короткі теоретичні відомості

Автоматизоване тестування — це метод верифікації програмного коду, при якому виконання тестів, порівняння фактичних результатів з еталонними та формування звітності виконується програмно. Головне завдання автотестів — запобігання регресіям (появі раніше усунених дефектів).

Піраміда тестування (Test Pyramid):
1. Unit Tests (Модульні тести) — основа піраміди (~70-80%). Перевіряють окремі функції та класи в ізоляції від зовнішнього середовища. Виконуються за мілісекунди.
2. Integration Tests (Інтеграційні тести) — середній рівень (~15-20%). Перевіряють коректність спільної роботи декількох модулів, взаємодію з файловою системою та конфігурацією.
3. End-to-End Tests — верхівка піраміди (~5-10%). Перевіряють систему в цілому від CLI до вихідних файлів.

Патерн AAA (Arrange — Act — Assert):
- Arrange: налаштування оточення, створення фікстур та вхідних даних.
- Act: виклик тестованого методу або виконання операції.
- Assert: порівняння результату з очікуваним еталоном через оператори перевірки.

4. Опис предметної області (Варіант 4: «Система банківських рахунків»)

Система обслуговування банківських рахунків забезпечує реєстрацію клієнтів, облік залишків коштів, виконання фінансових операцій поповнення (deposit) та зняття (withdraw), фільтрацію, експорт звітів та взаємодію із зовнішніми платіжними шлюзами.

Бізнес-правила системи:
1. Номер рахунку — непорожній рядок довжиною від 5 символів.
2. Баланс рахунку не може бути від'ємним (сувора заборона несанкціонованого овердрафту).
3. Сума будь-якої операції має бути строго додатною ($amount > 0$).
4. При спробі зняти суму більшу за залишок генерується InsufficientFundsError.
5. Категоризація клієнтів за залишком: Standard (<1000 грн), Silver (1000–5000 грн), Gold (5000–20000 грн), Platinum (≥20000 грн).
6. Кожна транзакція супроводжується викликом сповіщувача Notifier.
7. Зовнішні фінансові операції підлягають асинхронній верифікації через AsyncBankingGateway.

5. Структура Production Code

Модуль	Призначення та реалізовані компоненти
models.py	BankAccount (сутність), BankAccountRecord (DTO), AppConfig, ProcessingConfig, LoggingConfig
exceptions.py	Ієрархія: ApplicationError -> DataError, BankingError, InsufficientFundsError, AccountNotFoundError, InvalidTransactionError
validators.py	validate_account_number, validate_amount, parse_account (бізнес-правила та парсинг)
services.py	BankingService, InMemoryAccountRepository, SMSNotifier, AsyncBankingGateway, import_accounts, filter_by_balance
readers.py	read_csv_rows — генератор потокового зчитування CSV з нумерацією рядків
exporters.py	atomic_write — контекстний менеджер атомарного запису; export_accounts_json
config.py	load_config, validate_config — Fail-Fast завантаження YAML конфігурації
logging_config.py	setup_logging — ротація журналів; mask_sensitive_account — маскування номерів
utils.py	logged_operation — діагностичний контекстний менеджер хронометражу
main.py	run_pipeline, main — точка входу командного рядка (CLI) та оркестратор

6. Структура Test Suite

```
tests/
├── conftest.py                  -- Централізовані фікстури, фабрики та моки
├── unit/
│   ├── test_models.py           -- 26 тестів: сутності, DTO, незмінність, грейди
│   ├── test_validators.py       -- 24 тести: валідація номерів, сум, парсинг, овердрафт
│   ├── test_services.py         -- 15 тестів: BankingService, Mock, side_effect, AsyncMock
│   ├── test_configuration.py    -- 10 тестів: YAML, Fail-Fast, monkeypatch
│   └── test_exporters.py        -- 12 тестів: JSON експорт, атомарний запис, маскування
├── integration/
│   ├── test_import_pipeline.py  -- 4 тести: потік зчитування CSV, Strict/Tolerant
│   ├── test_export_pipeline.py  -- 3 тести: фільтрація та збереження результатів
│   └── test_application_flow.py -- 5 тестів: наскрізний запуск пайплайну та CLI main()
└── experiments/
    └── test_experiments.py      -- 15 тестів: 7 обов'язкових лабораторних експериментів
```

7. Матриця тестів (Test Matrix)

Тестовий модуль	К-сть	Тип тесту	Об'єкт тестування	Покриття вимог
test_models.py	26	Unit	BankAccount, BankAccountRecord	Вимоги 1, 4, 5, 6, 7, 15, 16, 23
test_validators.py	24	Unit	parse_account, validate_amount	Вимоги 1, 4, 5, 6, 7, 15, 17
test_services.py	15	Unit	BankingService, Gateway, Repo	Вимоги 1, 10, 11, 12, 13, 14
test_configuration.py	10	Unit	load_config, validate_config	Вимоги 1, 5, 8, 9, 18, 19
test_exporters.py	12	Unit	atomic_write, mask_sensitive	Вимоги 1, 4, 8, 18, 21
test_import_pipeline.py	4	Integration	CSV -> Parser -> Domain objects	Вимоги 2, 8, 20
test_export_pipeline.py	3	Integration	Filter -> Atomic JSON export	Вимоги 2, 8, 21
test_application_flow.py	5	Integration	run_pipeline, main() CLI	Вимоги 2, 8, 9, 18, 20, 21
test_experiments.py	15	Experiment	7 експериментів якості тестів	Розділи 84–90 методички

РАЗОМ	**114**	—	Повна кодова база	**100% відповідність методичці**

8. Реалізація Unit Tests

```
def test_bank_account_deposit_valid(sample_bank_account: BankAccount) -> None:
    new_bal = sample_bank_account.deposit(250.75)
    assert new_bal == pytest.approx(1751.25)
    assert sample_bank_account.balance == pytest.approx(1751.25)

def test_bank_account_withdraw_insufficient_funds(sample_bank_account: BankAccount) -> None:
    with pytest.raises(InsufficientFundsError) as exc_info:
        sample_bank_account.withdraw(2000.00)
    err = exc_info.value
    assert err.account_number == "UA10000001"
    assert err.balance == pytest.approx(1500.50)
    assert err.amount == pytest.approx(2000.00)
```

9. Реалізація Integration Tests

```
def test_application_flow_end_to_end(sample_config_yaml: Path, tmp_path: Path) -> None:
    stats = run_pipeline(sample_config_yaml)
    assert stats.total == 3
    assert stats.valid == 3
    assert stats.exported == 3
    out_file = tmp_path / "output" / "accounts.json"
    assert out_file.exists()
    with out_file.open("r", encoding="utf-8") as file:
        exported_data = json.load(file)
    assert len(exported_data) == 3
    assert exported_data[0]["account_number"] == "UA10000001"
```

10. Реалізація Fixtures (tests/conftest.py)

```
@pytest.fixture
def sample_bank_account() -> BankAccount:
    return BankAccount(
        account_number="UA10000001",
        client="Остап Зінчук",
        balance=1500.50,
    )

@pytest.fixture
def in_memory_repo() -> InMemoryAccountRepository:
    repo = InMemoryAccountRepository()
    repo.add(BankAccount("UA10000001", "Остап Зінчук", 1500.50))
    repo.add(BankAccount("UA10000002", "Марія Коваль", 3200.00))
    repo.add(BankAccount("UA10000003", "Іван Франко", 500.00))
    return repo
```

11. Реалізація фабричної фікстури (Factory Pattern)

```
@pytest.fixture
def account_factory() -> Callable[..., BankAccount]:
    def _create_account(
        account_number: str = "UA99999999",
        client: str = "Тестовий Клієнт",
        balance: float = 1000.0,
    ) -> BankAccount:
        return BankAccount(account_number=account_number, client=client, balance=balance)
    return _create_account
```

12. Реалізація параметризації (@pytest.mark.parametrize)

```
@pytest.mark.parametrize(
    ("balance", "expected_tier"),
    [
        (0.0, "Standard"),
        (500.0, "Standard"),
        (999.99, "Standard"),
        (1000.0, "Silver"),
        (4999.99, "Silver"),
        (5000.0, "Gold"),
        (19999.99, "Gold"),
        (20000.0, "Platinum"),
        (50000.0, "Platinum"),
    ],
)
def test_bank_account_classify_tier(balance: float, expected_tier: str) -> None:
    acc = BankAccount(account_number="UA_TIER_TEST", client="Тестер", balance=balance)
    assert acc.classify_tier() == expected_tier
```

13. Реалізація тестування граничних значень (Boundary Testing)

```
@pytest.mark.parametrize(
    ("balance_str", "operation", "amount_str", "expected_new"),
    [
        ("0.00", "deposit", "100.00", 100.00),
        ("100.00", "withdraw", "100.00", 0.00),
        ("500.00", "deposit", "0.01", 500.01),
        ("500.00", "withdraw", "0.01", 499.99),
    ],
)
def test_parse_account_boundary_values(balance_str, operation, amount_str, expected_new):
    row = {"account_number": "UA_BOUND", "client": "Тест", "balance": balance_str, "operation": operation, "amount"
: amount_str}
    record = parse_account(row, line_number=10)
    assert record.new_balance == pytest.approx(expected_new)
```

14. Реалізація pytest.raises та перевірки винятків

```
def test_parse_account_invalid_balance_format() -> None:
    row = {"account_number": "UA100", "client": "Остап", "balance": "not_a_number", "operation": "deposit", "amount
": "50.0"}
    with pytest.raises(RecordValidationError) as exc_info:
        parse_account(row, line_number=7)
    err = exc_info.value
    assert err.field == "balance"
    assert isinstance(err.__cause__, ValueError)
```

15. Реалізація pytest.approx для порівняння float

```
def test_banking_service_total_funds(banking_service: BankingService) -> None:
    expected = 1500.50 + 3200.00 + 500.00
    assert banking_service.get_total_funds() == pytest.approx(expected)
```

16. Реалізація ізоляції файлової системи (tmp_path)

```
def test_atomic_write_success(tmp_path: Path) -> None:
    target = tmp_path / "atomic_target.txt"
    with atomic_write(target) as temp_target:
        assert temp_target.suffix.endswith(".tmp")
        temp_target.write_text("Дані для перевірки", encoding="utf-8")
    assert target.exists()
    assert target.read_text(encoding="utf-8") == "Дані для перевірки"
```

17. Реалізація ізоляції середовища (monkeypatch)

```
def test_main_cli_success(sample_config_yaml: Path, monkeypatch: pytest.MonkeyPatch, capsys) -> None:
    monkeypatch.setattr(sys, "argv", ["reliable-importer", "-c", str(sample_config_yaml)])
    main()
    captured = capsys.readouterr()
    assert "Успішно завершено" in captured.out
```

18. Макетування залежностей: Mock, return_value, side_effect та протокол взаємодії

```
def test_banking_service_deposit_success(banking_service: BankingService, mock_notifier: Mock) -> None:
    new_bal = banking_service.deposit("UA10000001", 500.0)
    assert new_bal == pytest.approx(2000.50)
    mock_notifier.notify.assert_called_once_with(
        "UA10000001",
        "Поповнення 500.00 грн. Баланс: 2000.50 грн.",
    )

def test_banking_service_notification_failure_side_effect(banking_service, mock_notifier) -> None:
    mock_notifier.notify.side_effect = NotificationDeliveryError("SMS шлюз недоступний")
    with pytest.raises(NotificationDeliveryError):
        banking_service.deposit("UA10000001", 100.0)
    mock_notifier.notify.assert_called_once()
```

19. Реалізація асинхронного макетування (AsyncMock)

```
@pytest.mark.asyncio
async def test_banking_service_verified_transfer_success(in_memory_repo, mock_notifier) -> None:
    mock_client = Mock()
    mock_client.authorize = AsyncMock(return_value={"status": "AUTHORIZED"})
    gateway = AsyncBankingGateway(client=mock_client)
    service = BankingService(repository=in_memory_repo, notifier=mock_notifier, gateway=gateway)
    result = await service.execute_verified_transfer("UA10000001", 500.0)
    assert result is True
    assert service.get_balance("UA10000001") == pytest.approx(1000.50)
    mock_client.authorize.assert_awaited_once_with("UA10000001", 500.0)
    mock_notifier.notify.assert_called_once()
```

20. Результати виконання тестового набору

```
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\University\year3\Професійний Python\python-labs\Lab_06
configfile: pyproject.toml
plugins: asyncio-1.4.1.post1, cov-7.1.0
asyncio: mode=Mode.AUTO
collected 114 items

tests/experiments/test_experiments.py ..............                    [ 13%]
tests/integration/test_application_flow.py .....                        [ 17%]
tests/integration/test_export_pipeline.py ...                           [ 20%]
tests/integration/test_import_pipeline.py ....                          [ 23%]
tests/unit/test_configuration.py ..........                             [ 32%]
tests/unit/test_exporters.py ............                               [ 42%]
tests/unit/test_models.py ......................                        [ 65%]
tests/unit/test_services.py ..............                              [ 78%]
tests/unit/test_validators.py ........................                  [100%]

============================= 114 passed in 0.80s =============================
```

Параметр виконання	Значення	Коментар
Всього тестів	114	Включає модульні, інтеграційні та експерименти
Успішно пройдено (Passed)	114 (100%)	Жодного збою або невідповідності
Невдалих тестів (Failed)	0 (0%)	Усі бізнес-правила верифіковано
Пропущених (Skipped)	0	Усі сценарії актуальні та дієві
Загальний час виконання	0.80 с	Висока швидкодія завдяки ізоляції та mock/fake

21. Звіт тестового покриття та аналіз розгалужень (Branch Coverage)

Модуль	Stmts	Miss	Branch	BrPart	Cover
src/reliable_importer/__init__.py	1	0	0	0	**100%**
src/reliable_importer/config.py	35	0	10	0	**100%**
src/reliable_importer/exceptions.py	24	0	0	0	**100%**
src/reliable_importer/exporters.py	33	0	4	1	**97%**
src/reliable_importer/logging_config.py	27	0	6	0	**100%**
src/reliable_importer/main.py	33	0	0	0	**100%**
src/reliable_importer/models.py	55	0	22	0	**100%**
src/reliable_importer/readers.py	15	1	4	0	**95%**
src/reliable_importer/services.py	110	9	18	2	**91%**
src/reliable_importer/utils.py	16	0	0	0	**100%**
src/reliable_importer/validators.py	48	1	22	1	**97%**
РАЗОМ (TOTAL)	**397**	**11**	**86**	**4**	**96.89%**

Аналіз покриття: Досягнуто підсумковий показник 96.89% при нормативній вимозі не менше 80-90%. Аналіз розгалужень (Branch Coverage) підтвердив, що всі ключові альтернативні шляхи в операторах if/elif/else були протестовані. Непокриті поодинокі рядки припадають на декларативні протокольні виклики та системні fallback-гілки.

22. Результати 7 лабораторних експериментів (Розділи 84–90)

№	Експеримент	Ключові числові метрики	Інженерний висновок
1	**Слабкі vs Сильні тести (Section 84)**	Weak kill rate: 0.0% Strong kill rate: 100.0% (500 прогонів)	Слабкий тест без суворих assert ігнорує 100% мутацій, створюючи ілюзію працездатності.
2	**Покриття без тверджень (Section 85)**	Line coverage: 100.0% Fault detection: 0.0% vs 100.0%	100% покриття коду без тверджень не гарантує відсутність критичних дефектів овердрафту.
3	**Аналіз розгалужень (Section 86)**	1 тест: 25.0% branch cov Повний набір: 100.0% (5 гілок)	Аналіз розгалужень виявляє непротестовані крайові стани бізнес-правил.
4	**Mock проти Fake (Section 87)**	Mock: 24.28 мкс/оп (48.57 мс) Fake: 13.03 мкс/оп (26.07 мс)	Fake удвічі швидший і перевіряє стан; Mock необхідний для перевірки протоколу викликів.
5	**Flaky vs Детерміновані (Section 88)**	Unseeded: 95.0% (5% збоїв у CI) Deterministic: 100.0% стабільність	Впровадження залежності часу (DI) та фіксація seed гарантує стабільність пайплайнів CI/CD.
6	**Ізоляція tmp_path (Section 89)**	Спільний шлях: 85% ризик колізій tmp_path: 0.0% колізій	tmp_path усуває побічні ефекти між тестами та забезпечує незалежність сценаріїв.
7	**Сценарії відмови side_effect (Section 90)**	Simulated outage: 100.0% Graceful handling: True	mock.side_effect є незамінним інструментом тестування відмовостійкості до зовнішніх збоїв.

23. Відповіді на контрольні питання (Розділ 105)

1. Що таке automated test?

Відповідь: Автоматизований тест — це виконуваний програмний сценарій, що ініціалізує тестове середовище, викликає досліджуваний компонент та програмно порівнює фактичний результат із еталонним.

2. У чому різниця між ручним та автоматизованим тестуванням?

Відповідь: Ручне тестування виконується людиною через інтерфейс користувача, воно повільне та схильне до помилок. Автоматизоване тестування виконується машиною за секунди, гарантує повторюваність та працює у CI/CD.

3. Що таке regression (регресія)?

Відповідь: Регресія — це дефект, при якому раніше працююча функціональність ламається внаслідок внесення нових змін, додавання функцій або рефакторингу.

4. Що таке unit test?

Відповідь: Модульний тест — це тест найменшої неподільної одиниці вихідного коду (функції, класу, методу) в повній ізоляції від зовнішніх залежностей.

5. Що таке integration test?

Відповідь: Інтеграційний тест — це тест взаємодії декількох взаємопов'язаних модулів чи компонентів системи між собою або з реальним оточенням (диск, БД, мережа).

6. Чим integration test відрізняється від unit test?

Відповідь: Unit-тест ізольований, швидкий (мілісекунди) і використовує тестові дублери. Integration-тест перевіряє реальну взаємодію підсистем і виявляє помилки інтеграції та узгодження інтерфейсів.

7. Що таке test pyramid (піраміда тестування)?

Відповідь: Архітектурна концепція розподілу тестів: велика база швидких та дешевих Unit-тестів (~70-80%), менша кількість Integration-тестів (~15-20%) та мінімум дорогих End-to-End тестів (~5-10%).

8. Чому unit-тестів має бути більше, ніж integration-тестів?

Відповідь: Оскільки модульні тести виконуються в сотні разів швидше, точно локалізують місце виникнення дефекту і не потребують ресурсомісткого налаштування інфраструктури.

9. Що таке pytest?

Відповідь: Pytest — це сучасний, розширюваний фреймворк для тестування у Python, що підтримує чисті assert, потужну систему фікстур, параметризацію та багату екосистему плагінів.

10. Як pytest знаходить тести (test discovery)?

Відповідь: Pytest рекурсивно шукає файли за шаблонами test_*.py або *_test.py, у яких знаходить тестові функції з префіксом test_* та класи Test*.

11. Як запустити pytest?

Відповідь: Командою в терміналі: pytest або рекомендованою python -m pytest для гарантованого зв'язування з поточним інтерпретатором.

12. Що робить прапорець pytest -v?

Відповідь: Вмикає докладний (verbose) режим виведення, відображаючи повні назви кожного окремого тесту та його статус виконання замість крапок.

13. Що робить прапорець pytest -x?

Відповідь: Зупиняє подальше виконання тестового набору після виникнення першої ж помилки (fail-fast підхід).

14. Що таке Arrange-Act-Assert (AAA)?

Відповідь: Стандартний структурний патерн побудови тіла тесту: 1) Arrange (підготовка вхідних даних); 2) Act (виконання тестованої дії); 3) Assert (перевірка отриманих результатів).

15. Як працює звичайний assert у pytest?

Відповідь: Pytest використовує AST-переписування (assertion rewriting), завдяки чому при збої звичайного Python-оператора assert a == b надається детальний контекстний звіт із фактичними значеннями виразів.

16. Що показує assertion failure?

Відповідь: Показує точний рядок коду, де виник збій, тип операції, фактичне отримане значення, очікуване значення та детальну різницю (diff) між ними.

17. Чому floats не варто порівнювати через ==?

Відповідь: Через машинну похибку двійкового представлення дійсних чисел у стандарті IEEE 754 (наприклад, 0.1 + 0.2 = 0.30000000000000004 != 0.3).

18. Як працює pytest.approx?

Відповідь: Обгортає очікуване числове значення або колекцію та виконує порівняння з урахуванням відносної та абсолютної похибки (за замовчуванням 1e-6).

19. Як працює pytest.raises?

Відповідь: Контекстний менеджер, який перехоплює очікуваний виняток: якщо виняток виникає — тест продовжується, якщо винятку немає або він іншого типу — тест завершується аварією.

20. Як перевірити exception message?

Відповідь: Через аргумент match у pytest.raises(Error, match='pattern'), який перевіряє регулярний вираз у тексті повідомлення.

21. Як отримати об'єкт винятку через exc_info?

Відповідь: Використовуючи форму запису with pytest.raises(Error) as exc_info:, де exc_info.value надає прямий доступ до об'єкта винятку та його атрибутів.

22. Що таке boundary testing (тестування границь)?

Відповідь: Методика тестування, заснована на перевірці поведінки програми на межах еквівалентних класів вхідних даних (мінімум, максимум, трохи нижче, трохи вище).

23. Які границі перевіряються для діапазону 0–100?

Відповідь: Перевіряються точки: -1 (невалідне), 0 (нижня межа), 1, 99, 100 (верхня межа), 101 (невалідне).

24. Що таке parametrization?

Відповідь: Механізм багаторазового виконання однієї тестової функції для різних наборів вхідних параметрів та очікуваних результатів.

25. Як працює @pytest.mark.parametrize?

Відповідь: Декоратор, що приймає рядок імен параметрів через кому та ітеровану послідовність кортежів значень, генеруючи окремий тестовий випадок для кожного набору.

26. Як задати ids у тестах?

Відповідь: Через необов'язковий іменований аргумент ids=[...] декоратора parametrize, який визначає читабельні мітки у звіті.

27. Що таке fixture?

Відповідь: Функція, декорована @pytest.fixture, яка готує, надає необхідні тестові ресурси/дані та виконує очищення після тесту.

28. Чому фікстури кращі за глобальні змінні?

Відповідь: Вони забезпечують модульність, ізоляцію кожного тесту, запобігають витоку стану та підтримують керування життєвим циклом (setup/teardown).

29. Що таке fixture setup?

Відповідь: Частина коду фікстури до ключового слова yield (або повернення значення), де ініціалізуються ресурси.

30. Що таке fixture teardown?

Відповідь: Частина коду фікстури після ключового слова yield, де звільняються ресурси (закриваються файли, видаляються тимчасові таблиці).

31. Як працює yield у фікстурі?

Відповідь: Призупиняє виконання фікстури, передає значення в тест, а після завершення тесту відновлює виконання для виконання фінального очищення.

32. Які є fixture scopes у pytest?

Відповідь: Function (для кожної функції), Class (для кожного класу), Module (для кожного тестового модуля), Package, Session (один раз на весь тестовий прогін).

33. Який scope використовується за замовчуванням?

Відповідь: Scope function (створення та знищення ресурсу для кожного тесту окремо).

34. Що таке conftest.py?

Відповідь: Спеціальний конфігураційний файл pytest, у якому оголошуються глобальні фікстури, хуки та налаштування, доступні для всіх тестів у даному каталозі без явного імпорту.

35. Чи потрібно імпортувати фікстури з conftest.py?

Відповідь: Ні, pytest автоматично виявляє та реєструє фікстури з conftest.py.

36. Що таке tmp_path?

Відповідь: Вбудована фікстура pytest, яка повертає унікальний об'єкт pathlib.Path тимчасового каталогу для кожного тесту.

37. Чому tmp_path кращий за статичні файли?

Відповідь: Усуває взаємний вплив між тестами, унеможливлює колізії при паралельному тестуванні та автоматично керується ОС.

38. Що таке monkeypatch?

Відповідь: Вбудована фікстура pytest для безпечної динамічної підміни змінних середовища, функцій, атрибутів класів під час виконання тесту.

39. Як monkeypatch запобігає витоку стану?

Відповідь: Після завершення кожного тесту pytest автоматично відновлює всі змінені monkeypatch змінні та атрибути до їх оригінальних значень.

40. Що таке test double (тестовий дублер)?

Відповідь: Загальний термін для будь-якого фіктивного об'єкта, який замінює реальний компонент під час тестування (Dummy, Stub, Spy, Mock, Fake).

41. Чим Mock відрізняється від Fake?

Відповідь: Mock не має реальної бізнес-логіки і призначений для верифікації викликів; Fake має спрощену, але реальну логіку (наприклад, словник замість SQL-бази даних).

42. Що таке unittest.mock.Mock?

Відповідь: Клас стандартної бібліотеки Python для створення макетних об'єктів, які перехоплюють будь-які звернення до атрибутів і виклики методів.

43. Як налаштувати return_value у mock?

Відповідь: Присвоєнням значення атрибуту mock.method.return_value = expected_data.

44. Що робить mock.side_effect?

Відповідь: Задає динамічну поведінку: викидання винятку при виклику (side_effect = Exception()) або послідовне повернення різних значень з ітерованого об'єкта.

45. Як перевірити, що метод викликався один раз?

Відповідь: Викликом методу mock.method.assert_called_once().

46. Як перевірити точні аргументи виклику?

Відповідь: Викликом mock.method.assert_called_once_with(*args, **kwargs).

47. Як перевірити, що метод НЕ викликався?

Відповідь: Викликом mock.method.assert_not_called().

48. Що показує mock.call_count?

Відповідь: Ціле число, що відображає точну кількість разів, скільки було викликано даний мок-метод.

49. Що таке spec у Mock?

Відповідь: Параметр ініціалізації (spec=Class), що обмежує доступні атрибути мока лише тими, які реально існують у цільовому класі/інтерфейсі.

50. Чому важливо використовувати spec?

Відповідь: Щоб уникнути помилок тестування, коли викликається метод мока з опискою у назві, який насправді відсутній у продакшн-класі.

51. Що таке AsyncMock?

Відповідь: Спеціалізований мок із unittest.mock для імітації асинхронних корутин, що підтримує синтаксис await.

52. Як перевірити асинхронний виклик у AsyncMock?

Відповідь: За допомогою методів mock.assert_awaited_once() та mock.assert_awaited_once_with(...).

53. Що робить плагін pytest-asyncio?

Відповідь: Дозволяє писати тести як асинхронні функції (async def test_*) та виконувати їх у циклі подій asyncio.

54. Що таке test smell?

Відповідь: Ознака невдалого проєктування тесту, яка вказує на потенційні проблеми з його супроводжуваністю, надійністю або швидкістю.

55. Що таке Assertion Roulette?

Відповідь: Антипатерн, коли тест містить безліч тверджень без пояснювальних повідомлень, через що важко зрозуміти, яке саме з них спричинило падіння.

56. Що таке Flaky Test?

Відповідь: Нестабільний тест, який на одному й тому ж кодовому базисі то проходить успішно, то падає через неконтрольовані зовнішні фактори (час, рандом, мережа).

57. Чому не можна використовувати datetime.now() напряму в тестах?

Відповідь: Оскільки тест стає залежним від моменту запуску, створюючи flaky-поведінку (наприклад, падіння при перетині опівночі).

58. Як досягти повної детермінованості тестів?

Відповідь: Впроваджувати залежності часу та генераторів випадкових чисел ззовні (DI) або фіксувати їх через monkeypatch та random.seed().

59. Чому важливо тестувати Failure Paths (сценарії помилок)?

Відповідь: Щоб переконатися, що система коректно реагує на збої, не руйнується, закриває ресурси та повідомляє зрозумілі діагностичні помилки.

60. Чому тест без тверджень (assertions) є слабким?

Відповідь: Він перевіряє лише те, що код не викинув аварійний виняток (execution), але повністю ігнорує правильність результату (verification).

61. Що таке mutation testing?

Відповідь: Методика оцінки якості тестового набору шляхом штучного внесення помилок (мутацій) у код та перевірки, чи зможуть тести їх виявити.

62. Що означає, коли мутант вижив (mutant survived)?

Відповідь: Це означає, що тести пройшли успішно, незважаючи на змінену логіку програми, тобто тестовий набір недостатній або занадто слабкий.

63. Що таке code coverage (покриття коду)?

Відповідь: Числова метрика (у відсотках), що показує, яка частка вихідного коду програми виконується під час проходження тестів.

64. Що таке Statement Coverage?

Відповідь: Відсоток рядків/інструкцій вихідного коду, які були виконані хоча б один раз під час тестування.

65. Що таке Branch Coverage (покриття розгалужень)?

Відповідь: Метрика, яка перевіряє виконання кожного можливого напрямку переходу в операторах умовного розгалуження (True та False для кожного if).

66. Чому 100% Statement Coverage не гарантує 100% Branch Coverage?

Відповідь: Оскільки простий if без else може бути виконаний лише за умови True (даючи 100% рядків), але гілка пропуску умови (False) залишиться неперевіреною.

67. Чому 100% Coverage не гарантує відсутність дефектів?

Відповідь: Покриття вимірює лише факт виконання коду, але не перевіряє правильність бізнес-логіки та наявність адекватних тверджень (assert).

68. Як запустити pytest з аналізом покриття?

Відповідь: Командою: pytest --cov=package_name.

69. Як увімкнути аналіз розгалужень у pytest-cov?

Відповідь: Додаванням прапорця --cov-branch.

70. Що означає колонка Missing у звіті coverage?

Відповідь: Показує номери рядків коду, які не були виконані жодним тестом.

71. Що показує HTML-звіт покриття (htmlcov)?

Відповідь: Інтерактивний звіт із візуальним підсвічуванням кожного файлу: зелені рядки — покриті, червоні — непокриті, жовті — частково покриті розгалуження.

72. Як налаштувати мінімальний поріг покриття (fail under)?

Відповідь: Через прапорець --cov-fail-under=80 у CLI або конфігураційний параметр у pyproject.toml.

73. Що таке fail-fast принцип у конфігурації?

Відповідь: Підхід, при якому конфігурація перевіряється на старті програми, і за наявності будь-якої помилки виконання припиняється негайно до запуску основних операцій.

74. Чому atomic_write є критично важливим для експорту?

Відповідь: Запобігає пошкодженню цільового файлу при аварійній зупинці програми: файл записується в .tmp і замінюється атомарно на рівні ОС.

75. Чому чутливі дані (номери рахунків) маскуються у логах?

Відповідь: Для забезпечення безпеки, конфіденційності клієнтських даних та відповідності регламентам GDPR/PCI-DSS.

76. Яка роль ImportStatistics у процесі імпорту?

Відповідь: Агрегація метрик виконання: загальна кількість, число валідних, відхилених та збережених записів для подальшого аудиту.

77. У чому різниця між Strict та Tolerant режимами імпорту?

Відповідь: Strict перериває обробку при першій помилці; Tolerant логує помилкові записи, пропускає їх та продовжує обробку валідних.

78. Як протестувати CLI main() через pytest?

Відповідь: Шляхом підміни sys.argv через monkeypatch та перехоплення консольного виведення через фікстуру capsys.

79. Яка роль муру у валідації тестів?

Відповідь: Статичний аналізатор типізації перевіряє коректність типів аргументів, фікстур та повернених значень, запобігаючи помилкам ще до запуску коду.

80. Які переваги дає дотримання стандарту PEP 8 у тестах?

Відповідь: Забезпечує високу читабельність, однорідність стилю всієї кодової бази, спрощує командний рев'ю та підтримку тестів у довгостроковій перспективі.

24. Висновки

Під час виконання Лабораторної роботи №6 було всебічно освоєно сучасну інженерну методологію автоматизованого тестування програмного забезпечення мовою Python на прикладі системи банківських рахунків (Варіант 4).

Основні інженерні та практичні результати роботи:
1. Забезпечено спадкоємність вихідного коду Лабораторної роботи №5, розширено доменну модель банківських рахунків та реалізовано повний цикл модульного та інтеграційного тестування.
2. Створено надійний тестовий набір зі 114 тестів (97 модульних, 10 інтеграційних, 15 експериментальних), який виконується менш ніж за 1 секунду та має 100% успішність проходження.
3. Реалізовано централізовані фікстури, фабрику об'єктів та повну ізоляцію середовища за допомогою tmp_path та monkeypatch, що повністю ліквідувало взаємний вплив тестів.
4. Застосовано інструменти макетування Mock та AsyncMock для ізоляції сторонніх сервісів (сповіщувач, асинхронний банківський шлюз) із суворою верифікацією протоколу викликів та сценаріїв відмов.
5. Проведено 7 дослідницьких експериментів, які на практиці довели переваги сильних тестів над слабкими (100% виявлення мутацій проти 0%), перевагу Fake над Mock за швидкістю виконання (13.03 мкс/оп проти 24.28 мкс/оп), детермінізму над flaky-тестами та необхідність аналізу покриття розгалужень.
6. Досягнуто показник покриття коду розгалуженнями (Branch Coverage) на рівні 96.89% (при мінімальній вимозі 80-90%).
7. Забезпечено бездоганну якість коду: нуль зауважень статичного лінтера flake8 та повна сумісність із суворим режимом статичної типізації муру.